

SWE 4101

Object Oriented Concepts 1

Objectives

- Understand variables
- Identify types of variables
- Understand scope and lifetime
- Naming conventions in Java
- The different data types
- Conversion between types

✓ Why we need variables

- ✓ ● Programs need to store information
- ✓ ● Data changes during execution
- ✓ ● Variables allow us to:
 - ✓ ■ store
 - ✓ ■ update
 - ✓ ■ use data

What is a Variable?

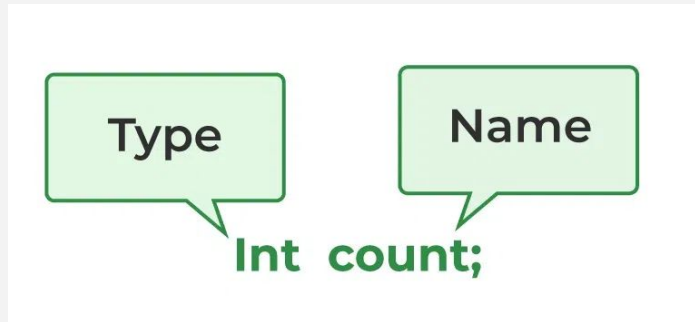
- ✓ • Variables are containers for storing data values
- ✓ • It is a **named memory location**
- ✓ • Stores a value
- ✓ • Has a data type

```
int age = 20;
```

This creates a space in memory called 'age' storing value 20

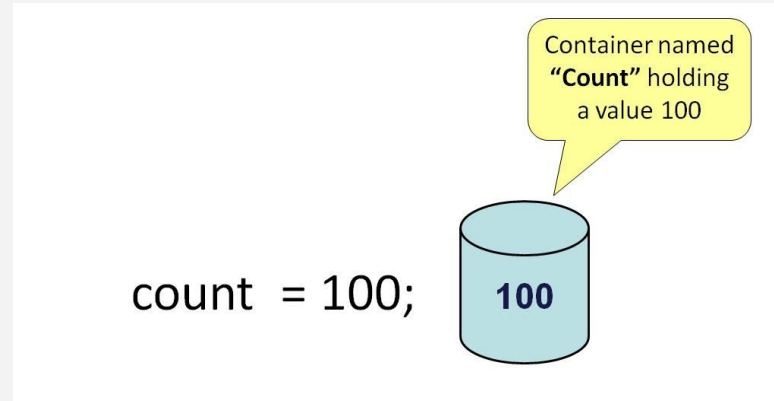
Variable Declaration

- ✓ Declaration tells the compiler the data type and the name of the variable without assigning any value
- ✓ Syntax: **dataType variableName;**



Variable Initialization

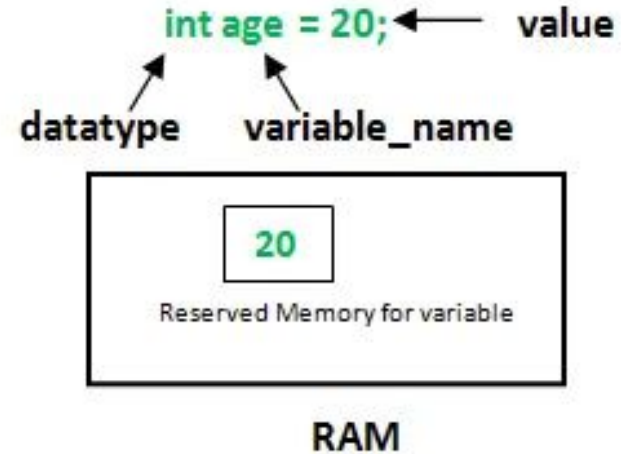
- ✓ Initialization means assigning a value to a variable after it has been declared
- ✓ Ensures the variable contains a valid value before it is used in the program
- ✓ Syntax: **variableName = value;**



Combined Declaration-Initialization

- You can perform both steps in a single statement for conciseness
- Syntax:

dataType variableName = value;



Variables in OOP

- Variables represent object state
- These define what information an object holds
- ✓ ● Types of Variables
 - ✓ ■ Local variables
 - ✓ ■ Instance variables
 - ✓ ■ Static variables

✓ Local Variables

- ✓ • These variables are declared and used within a specific method, constructor, or block of code
- ✓ • Only accessible within the scope in which they are defined
- ✓ • Do not get any default values
- ✓ • Local variables must be initialized before they can be used

```
void display() {  
    int x = 10;  
}
```

✓ Instance Variables

- ✓ These variables are declared within a class but outside any method, constructor, or block of code
- ✓ Associated with individual instances (objects) of the class
- ✓ Each object of the class has its own copy of the instance variables
- ✓ Instance variables are initialized to their default values if not explicitly assigned

✓ Instance Variable Example

```
class Student {  
    int id;  
    String name;  
}
```

```
Student s1 = new Student();  
Student s2 = new Student();
```

s1.name ≠ s2.name
s1.id ≠ s2.id

✓ Static Variables

- ✓ These variables are declared with the `static keyword` within a class but outside any method, constructor, or block of code
- ✓ Associated with the class itself rather than with objects
- ✓ Static variables are shared among all objects of the class
- ✓ Changes to the variable will be reflected in all objects
- ✓ Static variables are initialized to their default values if not explicitly assigned

Example

```
class Student {  
    String name;  
    int count = 0; // instance variable  
  
    Student(String name) {  
        this.name = name;  
        count++; // increments per object  
    }  
}
```

```
public class Main {  
    public static void main(String[] args){  
        Student s1 = new Student("Jane");  
        Student s2 = new Student("John");  
        Student s3 = new Student("Alex");  
  
        System.out.println(s1.count);  
        System.out.println(s2.count);  
        System.out.println(s3.count);  
    }  
}
```

Output:

1
1
1

Example

```
class Student {  
    String name;  
    static int count = 0; // shared  
  
    Student(String name) {  
        this.name = name;  
        count++; // shared increment  
    }  
}
```

```
public class Main {  
    public static void main(String[] args){  
        Student s1 = new Student("Jane");  
        Student s2 = new Student("John");  
        Student s3 = new Student("Alex");  
  
        System.out.println(Student.count);  
    }  
}
```

Output: 3

Combined Example

```
class Student {  
  
    // Instance variable (belongs to object)  
    String name;  
  
    // Static variable (belongs to class)  
    static int count = 0;  
  
    void display() {  
        // Local variable (inside method)  
        int marks = 90;  
  
        System.out.println("Marks: " + marks);  
    }  
}
```

✓ Scope and Lifetime

- ✓ ● The scope of a variable defines where the variable is accessible within a program
- ✓ ● As a general rule, variables that are defined within a block are not accessible outside that block
- ✓ ● The lifetime of a variable refers to how long the variable exists in memory before it is destroyed
- ✓ ● Destroying variables refers to deallocating the memory that was allotted to the variables when declaring it

Differences

Feature	Local	Instance	Static (Class)
Declared In	Method / constructor / block	Class (outside methods)	Class with static keyword
Scope	Only within that block	Entire class (access controlled separately)	Entire class
* Lifetime	During method execution	As long as object exists	Entire program
Default Value	No default value	Has default value	Has default value
Ownership	Belongs to method	Belongs to object	Belongs to class
* Key Feature	Must be initialized before use	Each object has its own copy	One shared copy for all objects

✓ Naming Conventions

- ✓ ● Set of guidelines for naming identifiers: classes, variables, methods, constants
- ✓ ● It directly affects how readable and maintainable your code is
- ✓ ● Not enforced by Java, but followed by all professional developers
- ✓ ● Improves readability, makes maintenance easier, reduces confusion, helps others understand your code faster

✓ Naming Classes

- ✓ Use **PascalCase** (each word capitalized)
- ✓ Should be **nouns**
- ✓ Should be meaningful
- ✓ Try to keep your class names simple and descriptive
- ✓ Use **whole words**
- ✓ **Avoid acronyms and abbreviations (unless the abbreviation is much more widely used than the long form, such as URL or HTML)**

```
class Student { }  
class BankAccount { }
```

✓ Naming Methods

- ✓ • Use camelCase
- ✓ • Should be verbs (actions)

```
calculateTotal()  
getName()  
displayInfo()
```

✓ Naming Variables

- ✓ Use camelCase
- ✓ Should be short but meaningful
- ✓ Avoid single-letter names (except temporary use)

```
int age;  
double totalMarks;  
String studentName;
```

- ✓ Single-letter names allowed only for short use

```
int i, j, k;  
char c;
```

- ✓ Should not start with underscore _ or dollar sign \$, even though both are allowed

✓ Naming Constants

- ✓ • Use ALL CAPITAL LETTERS
- ✓ • Words separated by _
- ✓ • Usually declared using final

```
final double PI = 3.14159;  
final int MAX_SIZE = 100;
```

✓ Data Types

- ✓ Define the kind of data a variable can hold and the memory required to store it
- ✓ Java is **statically typed** - type checking at compile-time
- ✓ Once you set a variable to a type, you cannot change it
- ✓ Typing is associated with the variable rather than the value

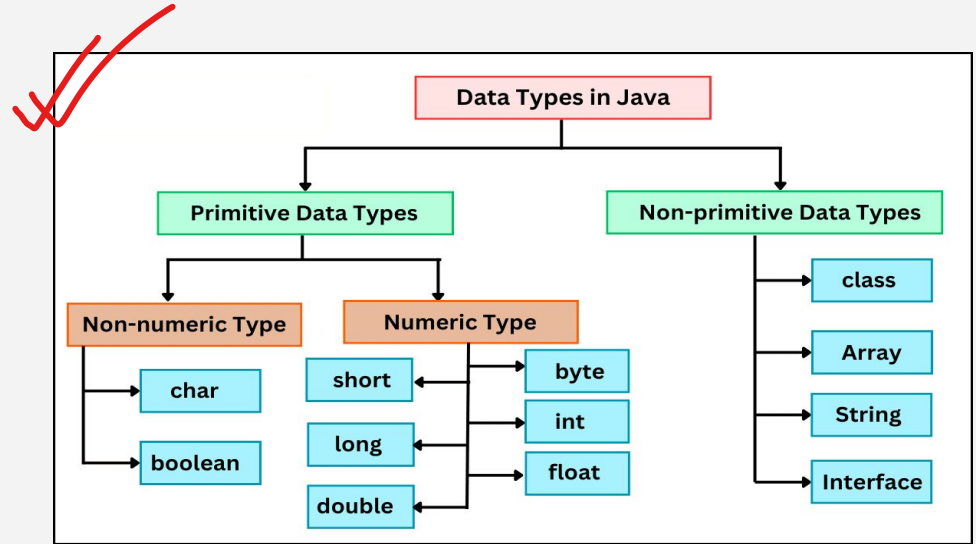
```
String str = "Hello";  
str = 5;
```

*// would throw an error since str is
// supposed to be a string only*

Data Types

* Two categories:

- ✓ Primitive Data Types
- ✓ Non-Primitive (Reference) Data Types



✓ Primitive Data Types

- ✓ ● Predefined by Java and named by reserved keywords
- ✓ ● Store values **directly** in memory
- ✓ ● Can hold exactly one value of its declared type at a time
- ✓ ● When another value is assigned to that variable, the new value replaces the previous one

✓ Primitive Data Types

Data Type	Default size	Default Value
boolean	1 bit	false
char	2 byte	'\u0000'
byte	1 byte	0
short	2 byte	0
int	4 byte	0
long	8 byte	0L
float	4 byte	0.0f
double	8 byte	0.0d

Primitive Data Types

```
int age = 20;
```

```
double cgpa = 3.75;
```

```
char grade = 'A';
```

```
boolean passed = true;
```

✓ Non-Primitive Data Types

- ✓ • Represent complex data and more sophisticated structures
- ✓ • Do **NOT** store the actual object/value directly
- ✓ • Store **references** to objects in memory
- ✓ • Point to where data resides rather than storing it directly
- ✓ • Examples - String, Arrays, Classes and Objects, etc.

```
String name = "John";
```

```
Student s1 = new Student();
```

✓ Non-Primitive Data Types

- ✓ • The default value for non-primitive data types is **null**, indicating that they point to no object in memory
- ✓ • Non-primitive types can have methods and fields

```
String name = "John";
```

```
name.length();  
name.toUpperCase();
```

Non-Primitive Data Types

- The default value for non-primitive data types is **null**, indicating that they point to no object in memory
- Non-primitive types can have methods and fields

```
int[] arr = {1, 2, 3};  
  
System.out.println(arr.length);
```

Non-Primitive Data Types

- The default value for non-primitive data types is **null**, indicating that they point to no object in memory
- Non-primitive types can have methods and fields

```
class Student {  
    public String name;  
    public int id;  
  
    public void display() {  
        System.out.println(name);  
    }  
}
```

```
Student s1 = new Student();  
s1.name = "John";  
s1.display();
```

✓ Why Primitives?

- ✓ ● Primitive types store values directly
- ✓ ● They are faster and more memory efficient
- ✓ ● Programs use huge numbers of integers, characters, and booleans
- ✓ ● Creating full objects for simple values would waste memory
- ✓ ● Java uses primitive types for performance

(Read this one)

Differences

Or Non-Primitive data type

PRIMITIVE DATA TYPES

VS

REFERENCE DATA TYPES

8

8 types

(byte, short, int, long, float, double, char, boolean)



Stores actual data

The variable holds the actual value.

1

Holds only one value

A primitive variable can store only one value at a time.



Uses less memory

Primitive types require less memory.



Faster

Operations on primitive types are faster.



Default values exist

They have default values.

∞

Unlimited types

(user-defined classes, arrays, Strings, etc.)



Stores a reference (address)

The variable holds the memory address of the actual object.



Can hold multiple values

A reference variable can refer to an object that contains multiple values.



Uses more memory

Reference types require more memory due to object overhead.



Slower

Operations on reference types are slower due to extra overhead.



Default value is null

Reference variables have the default value null.

✓Type Casting

- ✓● Used to convert primitive values from one data type to another
- ✓● Two types:
 - ✓1. Widening or Automatic Type Casting
 - ✓2. Narrowing or Explicit Casting

✓ Implicit Type Casting

- ✓ The conversion happens **automatically**
- ✓ Java automatically **converts a smaller compatible data type into a larger data type**
- ✓ No chance of data loss

```
int x = 10;  
double y = x;
```

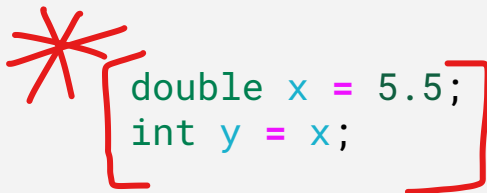
```
System.out.println  
n(y);  
// Prints 10.0
```

✓ Implicit Type Casting

```
char c = 'A';  
int x = c;  
  
System.out.println(x);  
// Prints 65
```

Works because characters have numeric Unicode values

internally



```
double x = 5.5;  
int y = x;
```

Compilation error! Needs explicit casting

✓ Explicit Type Casting

- ✓ • Conversion is done **manually** by programmer
- ✓ • Converts larger types into smaller types
- ✓ • Explicit cast is required
- ✓ • May cause data loss

```
double x = 5.5;  
int y = (int)x;
```

```
System.out.println(y);  
// Prints 5
```

✓ Explicit Type Casting

```
double d = 7.99;  
int x = (int)d;  
  
System.out.println(x);  
// Prints 7
```

Java does not round the number

It truncates it

✓ Examples

```
int x = 5;  
double y = x / 2;
```

```
System.out.println(y);  
// 2.0
```

```
System.out.println(5 / 2);  
// 2
```

Thank You